

TASK 6 :

LIST AND DICTIONARY COMPREHENSIONS

DESCRIPTION

Use list and dictionary comprehensions to create collections efficiently from existing data. This task explains how comprehensions simplify common Python data-processing operations such as transformation, filtering, cleaning, classification and creation of new collections.

OBJECTIVE

Learn concise Python techniques commonly used in data preprocessing and understand when comprehensions are appropriate compared with traditional loops.

TOOLS

- Python
- Jupyter Notebook

DELIVERABLES

- At least five list comprehension examples
- At least three dictionary comprehension examples
- Filtering examples
- Transformation examples
- Comparison with traditional loops
- Practical student and product data examples
- Interview questions and answers

KEY CONCEPT

A comprehension combines an expression, a loop over an iterable, and optionally a condition. It is especially useful when the required operation is simple and can be expressed clearly in one statement.

1. BASIC SYNTAX

List comprehension syntax:

```
[expression for item in iterable]
```

List comprehension with filtering:

```
[expression for item in iterable if condition]
```

Dictionary comprehension syntax:

```
{key_expression: value_expression for item in iterable}
```

Dictionary comprehension with filtering:

```
{key: value for item in iterable if condition}
```

2. BASIC LIST COMPREHENSION

The following example creates a new list containing the square of every number.

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

squares = [n ** 2 for n in numbers]

print("Original numbers:", numbers)
print("Squares:", squares)
```

Result concept: Each number is read from the original list and its square is placed into a new list.

3. FILTERING EVEN AND ODD NUMBERS

```
even_numbers = [n for n in numbers if n % 2 == 0]
odd_numbers = [n for n in numbers if n % 2 != 0]

print("Even numbers:", even_numbers)
print("Odd numbers:", odd_numbers)
```

The `if` condition controls which values are included in the resulting list.

4. STRING TRANSFORMATION

```
names = ["rahul", "priya", "aman", "neha"]

uppercase_names = [name.upper() for name in names]
name_lengths = [len(name) for name in names]

print("Uppercase names:", uppercase_names)
print("Name lengths:", name_lengths)
```

5. STUDENT MARKS FILTERING

Comprehensions can be used to select students or marks that satisfy a particular rule.

```
marks = [85, 92, 67, 76, 55, 88, 72]

passed_marks = [
    mark for mark in marks
    if mark >= 60
]

print("Passed marks:", passed_marks)
```

6. STUDENT GRADE CLASSIFICATION

A conditional expression can be placed inside a comprehension to classify values.

```
grades = [
    "A" if mark >= 80
    else "B" if mark >= 60
    else "C"
    for mark in marks
]

print("Grades:", grades)
```

7. FINDING HIGH-PERFORMING MARKS

```
high_marks = [
    mark for mark in marks
    if mark >= 80
]

print("Marks >= 80:", high_marks)
```

8. STRING FILTERING

```
subjects = [
    "Python", "SQL", "R",
    "Pandas", "AI", "Tableau"
]
```

```

]

long_subjects = [
    subject for subject in subjects
    if len(subject) > 3
]

print("Long names:", long_subjects)

```

9. CLEANING TEXT DATA

Comprehensions are useful for simple data-cleaning operations.

```

raw_names = [
    " rahul ", " PRIYA ",
    " aman ", " NEHA "
]

clean_names = [
    name.strip().title()
    for name in raw_names
]

print("Clean names:", clean_names)

```

10. NESTED LIST COMPREHENSION

Nested comprehensions can process values from nested collections.

```

matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

flattened = [
    value
    for row in matrix
    for value in row
]

print("Flattened matrix:", flattened)

```

11. PRICE CALCULATION

```

prices = [1000, 2500, 5000, 7500, 10000]

discounted_prices = [
    round(price * 0.90, 2)
    for price in prices
]

print("Original:", prices)
print("After 10% discount:", discounted_prices)

```

12. DICTIONARY COMPREHENSION — BASIC

Dictionary comprehension creates a dictionary using a key expression and a value expression.

```

square_dict = {
    n: n ** 2
    for n in range(1, 6)
}

print("Square dictionary:", square_dict)

```

13. DICTIONARY FROM TWO LISTS

```

names = ["Rahul", "Priya", "Aman", "Neha"]
marks = [85, 92, 67, 76]

student_scores = {

```

```

        name: mark
        for name, mark in zip(names, marks)
    }

    print("Student scores:", student_scores)

```

14. STUDENT GRADE DICTIONARY

```

students = {
    "Rahul": 85,
    "Priya": 92,
    "Aman": 67,
    "Neha": 76,
    "Rohit": 55
}

student_grades = {
    name: (
        "A" if mark >= 80
        else "B" if mark >= 60
        else "C"
    )
    for name, mark in students.items()
}

print("Student grades:", student_grades)

```

15. FILTERING A DICTIONARY

```

top_students = {
    name: mark
    for name, mark in students.items()
    if mark >= 80
}

print("Top students:", top_students)

```

16. STUDENT NAME LENGTH DICTIONARY

```

name_lengths = {
    name: len(name)
    for name in students
}

print("Name lengths:", name_lengths)

```

17. PRODUCT PRICE TRANSFORMATION

```

products = {
    "Laptop": 65000,
    "Phone": 30000,
    "Headphones": 2500,
    "Keyboard": 1800,
    "Monitor": 15000
}

discounted_prices = {
    product: round(price * 0.90, 2)
    for product, price in products.items()
}

print("Discounted prices:", discounted_prices)

```

18. PREMIUM PRODUCT FILTERING

```

premium_products = {
    product: price
    for product, price in products.items()
    if price >= 10000
}

print("Premium products:", premium_products)

```

19. PRACTICAL STUDENT RECORDS

A list of dictionaries is a simple way to represent tabular records in Python.

```
student_records = [
    {"name": "Rahul", "marks": 85},
    {"name": "Priya", "marks": 92},
    {"name": "Aman", "marks": 67},
    {"name": "Neha", "marks": 76},
    {"name": "Rohit", "marks": 55}
]

passed_students = [
    record["name"]
    for record in student_records
    if record["marks"] >= 60
]

score_dictionary = {
    record["name"]: record["marks"]
    for record in student_records
}

print("Passed students:", passed_students)
print("Score dictionary:", score_dictionary)
```

20. STUDENT PERFORMANCE TRANSFORMATION

```
performance = {
    record["name"]: (
        "Excellent" if record["marks"] >= 80
        else "Good" if record["marks"] >= 60
        else "Needs Improvement"
    )
    for record in student_records
}

print("Performance:", performance)
```

21. TRADITIONAL LOOP VS COMPREHENSION

Both approaches below create the same result. The comprehension is shorter, while the traditional loop can be easier to extend when the logic becomes complex.

```
# Traditional loop
loop_result = []

for n in numbers:
    if n % 2 == 0:
        loop_result.append(n ** 2)

# Comprehension
comprehension_result = [
    n ** 2 for n in numbers
    if n % 2 == 0
]

print("Loop:", loop_result)
print("Comprehension:", comprehension_result)
print("Same result:", loop_result == comprehension_result)
```

22. COMPARISON TABLE

Technique	Example Pattern	Purpose
List comprehension	[expr for x in data]	Create a new list
Filtered list	[expr for x in data if condition]	Select values

Dictionary comprehension	{k:v for x in data}	Create key-value data
Filtered dictionary	{k:v for x in data if condition}	Select entries
Traditional loop	for x in data: ...	Complex multi-step logic

INTERVIEW QUESTIONS AND ANSWERS

Q1. What is list comprehension?

A concise Python syntax for creating a list from an iterable, optionally using a condition.

Q2. How can you filter values using list comprehension?

Add an `if` condition after the iterable.

Q3. What is dictionary comprehension?

A concise syntax for creating dictionaries using key-value expressions.

Q4. Can comprehensions use conditions?

Yes. Conditions can filter items or conditional expressions can transform their values.

Q5. Can list comprehensions contain nested loops?

Yes, but excessive nesting can reduce readability.

Q6. Are comprehensions always faster than loops?

No. Their major benefit is concise and readable code for suitable operations.

Q7. Why are comprehensions useful in data preprocessing?

They provide a compact way to clean, transform and filter collections before analysis.

Q8. What is the difference between filtering and transformation?

Filtering selects items based on a condition, while transformation changes or calculates a value for each selected item.

COMPLETE FINAL PROGRAM

TASK 6 – LIST AND DICTIONARY COMPREHENSIONS
COMPLETE PROGRAM

```
numbers = list(range(1, 11))

squares = [n ** 2 for n in numbers]
even_numbers = [n for n in numbers if n % 2 == 0]
odd_numbers = [n for n in numbers if n % 2 != 0]

print("Squares:", squares)
print("Even:", even_numbers)
print("Odd:", odd_numbers)

# ----- STUDENT DATA -----
students = {
    "Rahul": 85,
    "Priya": 92,
    "Aman": 67,
    "Neha": 76,
    "Rohit": 55
}

passed = {
    name: mark
    for name, mark in students.items()
    if mark >= 60
}

grades = {
    name: (
        "A" if mark >= 80
        else "B" if mark >= 60
        else "C"
    )
    for name, mark in students.items()
}

print("Passed:", passed)
print("Grades:", grades)

# ----- PRODUCT DATA -----
products = {
    "Laptop": 65000,
    "Phone": 30000,
    "Headphones": 2500,
    "Keyboard": 1800,
    "Monitor": 15000
}

discounted = {
    product: round(price * 0.90, 2)
    for product, price in products.items()
}

premium = {
    product: price
    for product, price in products.items()
    if price >= 10000
}

print("Discounted:", discounted)
print("Premium:", premium)

# ----- STUDENT RECORDS -----
records = [
    {"name": "Rahul", "marks": 85},
    {"name": "Priya", "marks": 92},
    {"name": "Aman", "marks": 67},
    {"name": "Neha", "marks": 76},
    {"name": "Rohit", "marks": 55}
]
```

```
passed_students = [  
    r["name"] for r in records  
    if r["marks"] >= 60  
]  
  
score_dictionary = {  
    r["name"]: r["marks"]  
    for r in records  
}  
  
performance = {  
    r["name"]: (  
        "Excellent" if r["marks"] >= 80  
        else "Good" if r["marks"] >= 60  
        else "Needs Improvement"  
    )  
    for r in records  
}  
  
print("Passed students:", passed_students)  
print("Scores:", score_dictionary)  
print("Performance:", performance)
```